

Cryptography Self Project Report

I. Introduction / Background

Project title

ESP32 Password Vault — AES-GCM ciphertext on Firebase, keys stored on device

Project goal (overview)

Build a compact, demonstrable password manager using an ESP32 that encrypts user passwords with **AES-256-GCM** and uploads only the encrypted data (IV, tag, ciphertext) to Firebase Realtime Database. The AES key for each password is stored locally on the ESP32 (Preferences/NVS) under the password name (e.g., **Facebook**). The system supports simple web-based create/get operations and demonstrates end-to-end authenticated encryption and secure storage patterns suitable for a portfolio or coursework.

Problems & Proposed solution

- **Problem:** Cloud storage of plaintext or raw symmetric keys is unsafe. If the cloud is breached, plaintext or keys can be stolen.
- **Solution:** Store only ciphertext (with IV and authentication tag) in Firebase. Keep symmetric keys on the device. Use AES-GCM to provide both confidentiality and integrity. Provide a minimal web UI for convenience.

Motivation

- Learn and demonstrate practical AEAD cryptography on embedded hardware (ESP32).
- Combine IoT, cloud (Firebase), and crypto into a single reproducible project for a cybersecurity portfolio.
- Explore secure key handling tradeoffs on constrained devices.

Related cryptographic concepts

- **AES (Advanced Encryption Standard)** — block cipher family (AES-256 used here).
 - **GCM (Galois/Counter Mode)** — an AEAD mode that provides confidentiality (encryption) and authenticity (auth tag).
 - **IV / Nonce** — 12-byte random value for each AES-GCM encryption (must be unique per key).
 - **Authentication tag** — cryptographic tag appended to ciphertext that verifies integrity on decryption.
 - **Base64** — encoding used for storing binary data in JSON.
 - **NVS / Preferences** — ESP32 non-volatile storage used to save keys locally.
 - **HTTPS / REST** — secure transport layer used for communicating with Firebase.
-

II. System Design / Architecture

High-level components

1. ESP32 firmware (Arduino)

- Web server (small HTML forms)
- AES-GCM encrypt/decrypt using mbedTLS (bundled on ESP32)
- Key storage in Preferences (NVS)
- Firebase REST HTTP client for put/get JSON

2. Firebase Realtime Database

- Stores per-entry JSON objects under `/ciphertexts/{name}: {
 "ciphertext": "<B64>", "iv": "<B64>", "tag": "<B64>" }`

3. Client UI

- Browser UI served by ESP32 (create & get forms)

Data flow (short)

1. User opens `http://<esp-ip>/` in browser.
2. Create: user submits `name` and `password`.
 - ESP32: generate random key (32B) + random IV (12B).
 - Encrypt with AES-256-GCM → produce ciphertext + 16B tag.
 - Upload `{ciphertext, iv, tag}` to Firebase at `/ciphertexts/{name}.json`.
 - Store the base64 key locally in Preferences under key `name`.
3. Get: user requests `name`.
 - ESP32: fetch `/ciphertexts/{name}` from Firebase.
 - Retrieve local key for `name`.
 - Decrypt using AES-GCM. If tag verification fails, deny output.
 - Show plaintext result on the web page.

ASCII sequence diagram

```
Browser ---> ESP32 Web UI : POST /create (name,pwd)
ESP32 ---> Firebase (HTTPS): PUT /ciphertexts/{name} = {iv,tag,ciphertext}
ESP32 (local) : store key in Preferences[name]

Browser ---> ESP32 Web UI : GET /get?name={name}
ESP32 ---> Firebase (HTTPS): GET /ciphertexts/{name}
ESP32 (local) : read key from Preferences[name]
ESP32 (local) : AES-GCM decrypt(ciphertext, iv, tag, key)
ESP32 ---> Browser : show plaintext or error
```

III. Implementation Details

This section summarizes the major code elements so you can paste bits into the Implementation chapter of your report.

Files & structure (recommended)

```
/ESP32_PW_Vault
|-- main.ino
|-- controller/
|   |-- crypto.ino      // AES-GCM, base64, random helpers
|   |-- firebase.ino    // firebase_get, firebase_put
|   |-- storage.ino     // Preferences read/write helpers
|-- web/
|   |-- index.html      // served by ESP32 (or generated inline)
```

Key functions (what they do)

- `random_bytes(uint8_t *buf, size_t len)`
Uses `esp_fill_random()` to generate cryptographically secure random bytes.
- `base64_encode_buf()` / `base64_decode_to_buf()`
Wrap `mbedtls_base64_*` to convert binary ↔ Base64 for JSON transport.
- `aes256_gcm_encrypt(key, keylen, iv, ivlen, plain, plain_len, out_cipher, out_tag, taglen)`
Uses `mbedtls_gcm_crypt_and_tag()` to encrypt and generate auth tag (16B).
- `aes256_gcm_decrypt(key, keylen, iv, ivlen, cipher, cipher_len, tag, tag_len, out_plain)`
Uses `mbedtls_gcm_auth_decrypt()` to verify tag and decrypt. Returns bool for success.
- `firebase_put(path, json)` / `firebase_get(path)`
Uses `WiFiClientSecure` + `HTTPClient` to PUT/GET JSON to `https://<project>.firebaseio.com/<path>.json?auth=...`
- `prefs.putString(name, base64_key)` / `prefs.getString(name)`
Store/retrieve base64-encoded key in ESP32 NVS (`Preferences`).

Data structures / JSON example

Stored Firebase JSON under `/ciphertexts/Facebook`:

```
{
  "ciphertext": "BASE64_CIPHERTEXT",
  "iv": "BASE64_IV",
```

```
"tag": "BASE64_TAG"
}
```

Local Preferences entry for name **Facebook** (stored as base64 string):

```
prefs.begin("keys", false);
prefs.putString("Facebook", "BASE64_KEY");
prefs.end();
```

Libraries used (Arduino IDE)

- Built-in: `WiFi.h`, `WebServer.h`, `Preferences.h`, `esp_system.h`
- Built-in crypto: `bedtls/gcm.h`, `bedtls/base64.h`
- Optional / installed: `ArduinoJson` (for JSON manipulation)
- Standard: `HTTPClient.h`, `WiFiClientSecure.h`

Cryptographic choices & parameters

- **Cipher:** AES-256-GCM (AES block cipher with GCM AEAD mode)
- **Key size:** 32 bytes (256 bits)
- **IV/Nonce:** 12 bytes (96 bits), randomly generated per encryption
- **Tag:** 16 bytes (128 bits)
- **Encoding:** Base64 for IV / tag / ciphertext transfers over JSON
- **Storage policy:** Ciphertext → Firebase; Key → NVS (Preferences) on device

Important code-care items

- Always generate fresh random IV per encrypt (use `esp_fill_random()`).
- On decryption, if `aes256_gcm_decrypt()` returns false → **reject** and do not display plaintext.

- Zeroize key buffer after usage (e.g., `memset(key, 0, 32)`).
 - Use HTTPS for Firebase calls (`WiFiClientSecure`), avoid `setInsecure()` in production.
-

IV. Usage Guide

Dependencies & setup

1. Hardware

- ESP32 development board (DevKitC, WROOM, etc.)
- USB cable and a PC on the same network (Wi-Fi)

2. Software

- Arduino IDE (latest).
- Install ESP32 board support (Boards Manager → esp32 by Espressif).
- Library Manager: install `ArduinoJson` (v6).

3. Firebase

- Create a Firebase project; enable Realtime Database.
- Note your Realtime Database host (e.g., `myproj.firebaseio.com`) and configure DB rules for testing.
- Generate a database secret or use authenticated access token (recommended for production).

Deployment steps

1. Create a project folder and paste the provided sketch files (`main.ino` + controllers).
2. Edit configuration constants in `main.ino` (Wi-Fi SSID/PASS, `FIREBASE_HOST`, `FIREBASE_AUTH`).

3. Open Arduino IDE → select board **ESP32 Dev Module** → set correct COM port.
4. Compile & upload sketch.
5. Open Serial Monitor (115200) to get the IP address or view debug outputs.

Running the demo (example)

1. In a browser on the same network, open **http://<esp-ip>/**.
2. **Create** entry:
 - Name: **Facebook**
 - Password: **mysecret123**
 - Submit → ESP32 will:
 - generate key & iv, encrypt **mysecret123** with AES-GCM
 - upload **{ciphertext, iv, tag}** to Firebase under **/ciphertexts/Facebook**
 - store base64 key in Preferences as **Facebook**
3. **Get** entry:
 - Name: **Facebook**
 - Submit → ESP32 will:
 - fetch **/ciphertexts/Facebook** from Firebase
 - read key from Preferences
 - decrypt using AES-GCM → show plaintext if tag verifies

Example results (sample output)

- Firebase entry **/ciphertexts/Facebook.json**:

```
{  
  "ciphertext": "MEi2...==",  
  "iv": "3f5a...==",  
  "tag": "q9Xz...=="  
}
```

- ESP32 local Preferences `Facebook` = `"sGf7...=="` (base64 of 32 bytes)

Troubleshooting

- If decryption fails: check that the correct key is stored for that `name` and the firebase record hasn't been modified.
 - If Firebase PUT/GET fails: check Wi-Fi, `FIREBASE_HOST`, and `FIREBASE_AUTH`; inspect response from `HTTPClient`.
 - If sketch doesn't compile: ensure ESP32 board package installed and `ArduinoJson` present.
-

V. Conclusion and Future Work

Conclusion

The project demonstrates a minimal, practical ESP32 password vault that enforces authenticated encryption (AES-256-GCM). It separates key storage (device) from ciphertext storage (cloud), showing a sensible compromise for a student demo. The web interface makes it easy to test create/get flows and showcase the cryptographic design on a portfolio.

Limitations (that matter)

- Keys are stored unprotected in NVS — a physical attacker with hardware access could extract them.
- Current system uses a device-level local policy for keys; no user authentication is enforced on the web UI (anyone on LAN can call the endpoints).
- For a production system, server-side authentication, secure hardware (ATECC), and boot/flash protections are needed.

Future work and improvements

1. **Master-passphrase + KDF approach (recommended):** Derive per-entry keys from a user master passphrase using PBKDF2-HMAC-SHA256 (or Argon2) and per-entry salt; store only salt + ciphertext in Firebase (no local raw keys).
 2. **Use secure element (ATECC608A) or secure enclave:** move keys into a hardware-backed keystore so keys never leave the device.
 3. **Device hardening:** enable ESP32 Flash Encryption and Secure Boot to reduce key extraction risk.
 4. **Firebase security:** deploy proper Firebase Authentication (OAuth) & database rules to restrict access; use TLS and certificate pinning.
 5. **User authentication & UI:** add login, PIN/biometric unlock, rate-limiting / logout, and a nicer AJAX single-page UI.
 6. **Auditing & logging:** log operations securely with tamper-resistant audit records (append-only).
 7. **Backup / recovery:** secure export of keys or recovery via encrypted backup requiring multi-factor verification.
-

VI. References

(Use these references to fill the References section in your Word doc; include exact URLs and access dates if required by your academic format.)

- AES – FIPS 197: Advanced Encryption Standard.
- NIST SP 800-38D — Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM).
- mbedTLS documentation — GCM / AES usage.
- Firebase Realtime Database REST API documentation.
- Arduino ESP32 core documentation (WiFi, WebServer, Preferences).
- OWASP IoT Security and secure storage guidelines.

Appendix / Notes for report editing

- Replace placeholders (`FIREBASE_HOST`, `FIREBASE_AUTH`) with your actual values before testing.
- Add screenshots: (a) browser create/get UI, (b) Firebase JSON record, (c) Serial Monitor output showing IP and logs.
- Expand Implementation: paste snippets of your real code (crypto functions, firebase PUT/GET, Preferences read/write) under section III.
- Add a short Threat Model subsection if your course requires it (who you defend against: remote attacker, local attacker, physical attacker).